

第8章 NoSQL | 开卷速查版

本章一句话： **NoSQL** 是为 Web 2.0、高并发、海量非结构化/半结构化数据和水平扩展而出现的一类数据库。它通常放弃严格关系模型和部分强事务能力，用更灵活的数据模型和分布式架构换取**高可用、高扩展、最终一致性**。🌙

常考题型： NoSQL 是什么；四种 **Data Model**；ACID；CAP 理论；CAP 里的 C 和 ACID 里的 C 区别；强一致性、线性一致性、最终一致性；BASE 原则；为什么 NoSQL 常牺牲一致性换可用性。

翻页关键词： NoSQL定义 → Data Model → Key-value → Document → Column-family → Graph → ACID → CAP → 一致性定义 → BASE → 最终一致性。

1. NoSQL 是什么

NoSQL 可以理解为 **Not Only SQL**，不是“完全不要 SQL”，而是指一类不以传统关系模型为核心、面向分布式扩展和灵活数据模型的数据库。

PPT里说 NoSQL 很难严格定义，但多数 NoSQL 数据库有这些特点：**非关系型、开源、基于集群而非单机、Schema-less、具有不同 data models**。

为什么出现 NoSQL？ 传统关系型数据库适合二维表、SQL、事务，但在互联网高并发、海量数据、灵活数据结构、水平扩展场景下会遇到压力。NoSQL 的出现是为了支持更高并发、更大规模、更灵活的数据模型。

考试写法： NoSQL 是一类面向高并发、海量数据和分布式扩展需求的非关系型数据库。它通常不强依赖固定 Schema 和关系模型，而是采用键值、文档、列族、图等数据模型，并在 CAP 约束下常通过牺牲部分强一致性来换取高可用性和水平扩展能力。

2. NoSQL 诞生背景和动机

PPT里把大数据与 NoSQL 的驱动因素概括为：**Web 2.0、高并发、海量非结构化数据**。社交网络、日志、用户生成内容大量增长，传统数据库的单机架构、复杂事务机制和垂直扩展成本难以满足需求。

主要动机：

高并发读写： 电商、社交、日志系统可能有百万级并发，传统数据库锁、事务、复杂查询可能成为瓶颈。

海量数据存储： 数据量从 GB 到 TB/PB，单机数据库难以承载。

水平扩展： 通过增加普通机器扩容，而不是不断升级昂贵单机。

灵活数据模型： JSON、日志、图关系、时间序列等数据不一定适合二维表。

CAP 取舍： 分布式系统中网络分区不可避免，很多系统选择牺牲强一致性，换取可用性和分区容忍性。

考试写法： NoSQL 出现的背景是 Web 2.0 应用带来的高并发读写、海量非结构化数据和水平扩展需求。传统关系型数据库强调固定 Schema、SQL 和 ACID 事务，在分布式大规模场景下扩展成本高、性能压力大，因此 NoSQL 采用更灵活的数据模型和分布式架构，并常用最终一致性换取高可用性。

3. Data Model 总览

PPT中 NoSQL 的数据模型主要分为四类：

```
Key-value
Document
Column family
Graph
```

也就是：**键值模型、文档模型、列族模型、图模型。**

可以先用一句话记：

```
Key-value: 按 key 找 value
Document: 按 JSON 文档存复杂对象
Column-family: 按 RowKey 管海量稀疏表
Graph: 按点和边表达关系
```

模型	数据结构	适合场景	典型系统
Key-value	key -> value	缓存、Session、购物车	Redis、DynamoDB
Document	JSON / BSON 文档	用户资料、内容管理、订单详情	MongoDB、CouchDB
Column-family	RowKey + 列族 + 列	日志、时序、稀疏大表	HBase、Cassandra
Graph	点 + 边 + 属性	社交关系、推荐、路径查询	Neo4j

4. Key-value Store

Key-value store 是最简单的 NoSQL 模型。它把数据存成：

```
key -> value
```

例如：

```
"user:1001" -> "{name:'Alice', age:20}"
"session:abc" -> "{user_id:1001, expire:...}"
```

系统主要根据 key 做快速读写，不关心 value 里面具体结构。value 可以是字符串、JSON、二进制对象等。

优点：模型简单，读写快，容易分布式扩展。**缺点：**查询能力弱，通常只能按 key 查；不适合复杂条件查询和 join。

适合场景：缓存、Session、购物车、计数器、排行榜、简单状态存储。

考试写法：Key-value 模型用唯一 key 映射 value，系统主要通过 key 进行快速读写。它结构简单、性能高、易扩展，适合缓存、Session 和购物车等场景，但不擅长复杂条件查询和关系运算。

5. Document Model

Document model 把一条数据存成一个文档，通常类似 JSON。

例子：

```
{
  "user_id": "1001",
  "name": "Alice",
  "age": 20,
  "address": {
    "city": "Shanghai",
    "district": "Yangpu"
  },
  "tags": ["student", "basketball"]
}
```

它适合存嵌套结构，因为一个对象的多层属性可以直接放在一个文档里，不必像关系数据库那样拆成多张表再 join。

优点：Schema 灵活，适合半结构化数据；一个业务对象可以整体存取。**缺点：**数据冗余可能增加；跨文档 join 和复杂事务较弱。

适合场景：用户资料、文章内容、商品详情、订单信息、日志 JSON。

考试写法：Document 模型以 JSON 或类似 JSON 的文档为基本存储单位，适合表达嵌套和半结构化数据。它 Schema 灵活，可以把一个业务对象整体存储和读取，适合用户资料、内容管理、商品详情等场景，但跨文档 join 和复杂事务能力通常较弱。

6. Column-family Model

Column-family model 是列族模型，典型系统是 **HBase / Cassandra**。它不是普通关系数据库的“列”，而是按 **RowKey + Column Family + Column** 组织海量稀疏数据。

可以写成：

```
RowKey -> ColumnFamily -> Column -> Value
```

结合 HBase：

```
(RowKey, ColumnFamily:Qualifier, Timestamp) -> Value
```

例子：

```
rowkey = user_1001
info:name -> Alice
info:age -> 20
post:p001 -> hello
```

优点： 适合海量稀疏表，按 RowKey 随机读写快，易水平扩展。**缺点：** 复杂 SQL、join、二级索引能力弱；表设计强依赖访问模式。

适合场景： 日志数据、时间序列、监控数据、Feed 流索引、搜索索引、用户行为记录。

考试写法： Column-family 模型用 RowKey 定位行，再通过列族和列定位具体值，适合海量稀疏数据和按 RowKey 的高并发读写。HBase 就是典型列族数据库，适合日志、时序数据、搜索索引等场景，但不擅长复杂 SQL 和 join。

7. Graph Model

Graph model 用图来表示数据，核心是：

```
Vertex / Node: 点
Edge: 边
Property: 点或边上的属性
```

例子：

```
(Alice) -[friend]-> (Bob)
(Alice) -[likes]-> (Movie_1)
(Bob) -[works_at]-> (Company_X)
```

它适合表达关系密集的数据。如果用关系数据库查“朋友的朋友”“两个人之间最短路径”，可能需要很多 join；图数据库直接沿边遍历。

优点： 适合关系查询、路径查询、推荐。**缺点：** 不一定适合大规模简单 key-value 读写；分布式图查询也可能复杂。

适合场景： 社交网络、知识图谱、推荐系统、路径分析、欺诈检测。

考试写法： Graph 模型用点、边和属性表示数据，适合表达复杂关系和路径查询。它在社交网络、推荐系统、知识图谱等场景中比关系数据库的多表 join 更自然，但不一定适合简单大规模键值读写。

8. 四种 Data Model 怎么选

考试看到场景题，可以按这个规则选：

只按 key 快速读写 -> Key-value
一个对象有嵌套 JSON -> Document
海量稀疏表、RowKey 查询 -> Column-family
关系和路径最重要 -> Graph

例子：

缓存用户登录态：Key-value。存商品详情 **JSON**：Document。存传感器时序数据：Column-family。查社交网络二度好友：Graph。存 **Feed 流 inbox/outbox** 索引：Column-family。存文章内容和评论嵌套结构：Document。

考试写法：选择 NoSQL 数据模型时应根据访问模式决定。如果主要按 key 读写，选择 Key-value；如果数据天然就是 JSON 或嵌套对象，选择 Document；如果是海量稀疏表并按 RowKey 访问，选择 Column-family；如果核心是关系和路径遍历，选择 Graph。

9. ACID 特性

ACID 是传统数据库事务的经典特性。PPT里列出四个：**Atomicity**、**Consistency**、**Isolation**、**Durability**，并说明 ACID 中的一致性也叫事务一致性。

A: Atomicity，原子性 事务要么全部成功，要么全部失败。不能只做一半。

C: Consistency，一致性 / 事务一致性 事务执行前后，数据库都必须处于合法状态。比如余额不能凭空增加，外键约束不能被破坏。

I: Isolation，隔离性 并发事务之间互不干扰，看起来像一个一个执行。

D: Durability，持久性 事务提交后，结果要永久保存，系统崩溃后也不能丢。

考试写法：ACID 是传统数据库事务的四个特性：原子性表示事务要么全成功要么全失败；一致性表示事务将数据库从一个合法状态转移到另一个合法状态；隔离性表示并发事务互不影响；持久性表示事务提交后结果永久保存。

10. ACID 里的 C 和 CAP 里的 C 不一样

这个非常容易考。

ACID 里的 Consistency：事务一致性。强调事务执行前后，数据库约束不被破坏。比如转账前后总金额不变。

CAP 里的 Consistency：分布式读写一致性。强调一个写操作成功后，后续读操作必须读到这个最新值。PPT里 CAP 的 C 定义为：写完成后开始的任何读操作都必须返回该值。

对比：

C	所属理论	含义
ACID 的 C	事务理论	数据库从合法状态到合法状态

C	所属理论	含义
CAP 的 C	分布式系统理论	写成功后，后续读必须看到最新值

考试写法：ACID 中的一致性是事务一致性，强调事务前后数据库满足完整性约束；CAP 中的一致性是分布式读写一致性，强调写操作完成后，后续读操作必须读到最新写入结果。二者语境不同，不能混淆。

11. CAP 理论

CAP 是分布式系统中的三要素：

C: Consistency, 一致性 写操作成功后，后续读必须读到最新值。

A: Availability, 可用性 每个请求都能得到响应，系统服务可用。

P: Partition Tolerance, 分区容忍性 网络分区、节点之间通信故障时，系统仍能继续工作，不返回错误结果。

PPT里说明：CAP 原理中，分布式系统最多只能同时实现三者中的两个，不可能三者兼顾；对于分布式数据系统，分区容忍性是基本要求；多数 Web 应用会牺牲一致性换取高可用性。

考试写法：CAP 理论指出，分布式系统中的一致性、可用性和分区容忍性三者最多只能同时满足两个。由于网络分区在分布式系统中不可避免，P 通常是基本要求，因此系统往往需要在 C 和 A 之间取舍。很多 NoSQL 系统选择牺牲强一致性，换取高可用性和分区容忍性。

12. CAP 三类取舍：CP、AP、CA

CP：一致性 + 分区容忍性。 网络分区时，系统宁愿拒绝部分请求，也要保证读到的是一致数据。适合强一致要求高的场景。

AP：可用性 + 分区容忍性。 网络分区时，系统继续响应请求，但不同副本可能暂时不一致，之后再慢慢收敛。很多 NoSQL 偏 AP。

CA：一致性 + 可用性。 在没有网络分区的单机或局域环境里比较容易；但真正分布式系统中 P 很难放弃，所以 CA 在分布式容错场景下通常不现实。

考试写法：在真实分布式系统中，网络分区不可避免，因此 P 通常必须满足。CP 系统在分区时优先保证一致性，可能牺牲部分可用性；AP 系统在分区时优先保证可用性，允许短暂不一致并依靠最终一致性收敛。很多 Web 场景下的 NoSQL 系统更偏 AP。

13. 一致性的若干定义

PPT中区分了单机环境下的事务一致性，以及分布式环境下的强一致性和线性一致性。

事务一致性

这是 ACID 里的 C，属于单机/数据库事务语境。意思是一个事务必须把数据库从一个合法一致状态转移到另一个合法一致状态。

例子：转账 100 元，A 扣 100，B 加 100，总金额不变。

强一致性

工程化定义：一旦写操作成功，之后任何读操作都必须立即看到最新写入的数据。PPT里也这样定义强一致性。

例子：用户修改密码成功后，任何节点上的登录校验都必须立刻使用新密码。

线性一致性

更严格、更形式化。每个操作都像是在调用开始和返回结束之间的某个瞬间原子完成，并且满足真实时间顺序。PPT中英文定义是：Every operation appears to occur atomically at some point between its invocation and its response。

简单理解：系统虽然是分布式的，但表现得像一个单机原子对象。

最终一致性

如果没有新的更新，经过一段时间后，所有副本最终会达到一致。PPT里定义：当长时间没有新的更新发生时，所有更新最终会传播到整个系统中，所有节点最终保持一致。

考试写法：一致性有不同语境。ACID 中的一致性是事务一致性，强调事务前后数据库状态合法；强一致性强调写成功后任何后续读都立即看到最新值；线性一致性是更严格的形式化定义，要求操作像在调用和返回之间某一瞬间原子发生；最终一致性允许短暂不一致，但在没有新更新后，各副本最终收敛一致。

14. 强一致性 vs 最终一致性

这个很可能考“区分强和最终”。

对比点	强一致性	最终一致性
写成功后读	立即读到最新值	可能短时间读到旧值
副本状态	要求立刻一致	允许暂时不一致
可用性	故障/分区时可能降低	可用性更高
延迟	往往更高	往往更低
适合场景	金融、库存、密码修改	社交点赞、评论数、缓存、Feed

例子：

强一致性例子：银行卡转账后余额必须准确，不能一个节点看到扣款、另一个节点没看到。

最终一致性例子：朋友圈点赞数短时间显示 99 或 100 都可以接受，只要最后一致。

考试写法：强一致性要求写操作成功后所有后续读立即看到最新值，适合金融交易、库存扣减等不能容忍旧数据的场景。最终一致性允许系统在短时间内读到旧值或副本不一致，但在没有新更新后所有副本最终收敛一致，适合社交网络、缓存、Feed 流等对短暂不一致容忍度较高的场景。

15. BASE 原则

BASE 是高可用分布式系统里常见的实践原则，是对 ACID 强一致性的放松。PPT里说 BASE 是对 CAP 定理的实践性妥协，包括 **Basically Available**、**Soft State**、**Eventually Consistent**。

BA: Basically Available，**基本可用** 系统故障时仍提供核心功能，但可能降级。比如返回缓存数据、简化服务、部分功能不可用。

S: Soft State，**软状态** 系统状态可以暂时不一致，因为异步复制或延迟传播，状态会变化，不要求每一刻都强一致。

E: Eventually Consistent，**最终一致性** 如果没有新的更新，经过一段时间后，各副本最终会一致。

考试写法：BASE 是高可用分布式系统中对 ACID 的放松。Basically Available 表示系统在故障时仍提供核心服务但可能降级；Soft State 表示系统允许中间状态暂时不一致；Eventually Consistent 表示在没有新更新的情况下，经过一段时间后所有副本最终一致。

16. ACID vs BASE

对比点	ACID	BASE
核心目标	强事务、强一致	高可用、高扩展
一致性	强一致 / 事务一致	最终一致
可用性	故障时可能拒绝请求	尽量提供核心服务
状态	每一步尽量严格一致	允许软状态
适合	金融、订单强事务	社交、日志、缓存、Feed
系统风格	传统 RDBMS	大规模 NoSQL / 分布式系统

考试写法：ACID 强调事务的原子性、一致性、隔离性和持久性，适合对一致性要求高的传统事务场景；BASE 强调基本可用、软状态和最终一致性，适合高并发、高可用、可扩展的分布式系统。NoSQL 常采用 BASE 思想，在可接受短暂不一致的前提下提升系统可用性和扩展性。

17. 最终一致性 Eventual Consistency

最终一致性可以单独拿出来背，因为 PPT 标了重点。

定义：当长时间没有新的更新发生时，所有更新最终都会传播到整个系统中，并且所有节点最终保持一致。PPT还补充说，对于某个已接受的更新 data 和某个给定节点 node，最终会出现两种情况之一：更新传播到该节点，或该节点被删除。

简单理解：

现在可能不一致
过一会儿同步完成
最后大家一致

例子：

用户点赞一条微博
节点 A 显示 100 个赞
节点 B 暂时显示 99 个赞
过一段时间复制完成后，A/B 都显示 100

考试写法：最终一致性允许分布式副本在短时间内不一致，但如果没有新的更新发生，已经被接受的更新最终会传播到所有仍然存在的副本节点，使所有副本最终达到一致状态。它常用于对短暂不一致可容忍、但要求高可用和高扩展的 NoSQL 系统。

18. 为什么 NoSQL 常牺牲一致性换可用性

分布式系统中，机器可能宕机，网络可能分区。此时系统面临选择：

要一致性：可能拒绝请求，牺牲可用性
要可用性：继续响应请求，允许暂时不一致

很多互联网应用更关心“服务不能挂”。比如点赞数、评论数、Feed 排序短时间不一致通常可以接受；但系统不可用，用户就完全不能使用。

所以很多 NoSQL 偏向：

P：必须要
A：尽量要
C：放松为最终一致性

也就是偏 **AP + BASE**。

考试写法：在分布式系统中，网络分区是必须考虑的情况，因此 P 通常不能放弃。当分区发生时，如果系统坚持强一致性，可能需要拒绝部分请求，降低可用性；而许多 Web 应用更重视持续服务，因此 NoSQL 常选择牺牲强一致性，采用最终一致性和 BASE 原则来换取高可用和可扩展性。

19. MPP vs NoSQL

PPT中有 MPP 数据库和 NoSQL 的对比。MPP 更偏 **大规模数据分析 OLAP**，NoSQL 更偏 **高并发读写和灵活数据模型**。

对比点	MPP 数据库	NoSQL 数据库
设计目标	大规模数据分析、复杂查询、高吞吐计算	高并发读写、灵活模式、海量存储
数据类型	结构化数据为主	半结构化、非结构化、键值/文档/列族/图
查询能力	SQL、复杂分析、聚合、join	按模型访问，复杂查询较弱
一致性	通常更接近 ACID / 强一致	常遵循 BASE / 最终一致
扩展方式	分布式并行查询	天然水平扩展、分片、副本
典型场景	数据仓库、BI、OLAP	缓存、日志、社交、时序、Feed

考试写法： MPP 数据库主要面向结构化数据的大规模 OLAP 分析，强调复杂 SQL 查询和高吞吐并行计算；NoSQL 数据库主要面向高并发读写、灵活数据模型和海量数据存储，常采用 BASE 和最终一致性，适合 Web 2.0、日志、社交和时序等场景。

20. NoSQL 和 NewSQL 简要区分

PPT里提到 **NewSQL** 是为了弥补 NoSQL 的 ACID 缺失，把分布式扩展能力和 SQL/事务结合起来。

NoSQL： 更灵活、更高可用，常牺牲部分强事务能力。**NewSQL：** 希望既能像 NoSQL 一样水平扩展，又能支持 SQL 和强事务。

典型 NewSQL：Spanner、CockroachDB、TiDB、Aurora。

考试一般不展开，知道方向即可。

考试写法： NoSQL 主要通过灵活数据模型和最终一致性支持高可用与扩展；NewSQL 则试图结合传统关系数据库的 SQL/事务能力与分布式系统的水平扩展能力，用来弥补 NoSQL 在 ACID 事务方面的不足。

本章高频简答模板

1. NoSQL 是什么？ NoSQL 是一类面向高并发、海量数据和分布式扩展需求的非关系型数据库。它通常具有 Schema-less、集群化、水平扩展和多样数据模型等特点，常采用最终一致性和 BASE 原则来换取高可用性和扩展性。

2. NoSQL 的四种数据模型是什么？ NoSQL 常见数据模型包括 Key-value、Document、Column-family 和 Graph。Key-value 适合按 key 快速读写；Document 适合 JSON 等半结构化对象；Column-family 适合海量稀疏表和 RowKey 查询；Graph 适合关系和路径查询。

3. ACID 是什么？ ACID 是事务的四个特性：Atomicity 原子性表示事务要么全成功要么全失败；Consistency 一致性表示事务前后数据库状态合法；Isolation 隔离性表示并发事务互不干扰；Durability 持久性表示提交后的结果永久保存。

4. CAP 理论是什么？ CAP 理论指出，分布式系统中 Consistency、Availability、Partition Tolerance 三者最多只能同时满足两个。由于网络分区在分布式系统中不可避免，P 通常必须考虑，因此系统往往需要在一致性和可用性之间取舍。

5. ACID 的 C 和 CAP 的 C 有什么区别？ ACID 的 C 是事务一致性，强调事务把数据库从一个合法状态转移到另一个合法状态；CAP 的 C 是分布式读写一致性，强调写操作成功后，后续读操作必须读到最新写入值。

6. 强一致性和最终一致性有什么区别？ 强一致性要求写操作成功后，所有后续读立即看到最新值；最终一致性允许系统短时间内存在副本不一致，但在没有新更新后，所有副本最终会收敛一致。强一致性适合金融、库存等场景；最终一致性适合社交、缓存、Feed 等可容忍短暂不一致的场景。

7. BASE 原则是什么？ BASE 包括 Basically Available、Soft State、Eventually Consistent。基本可用表示系统故障时仍提供核心服务但可能降级；软状态表示系统允许临时不一致；最终一致性表示没有新更新后各副本最终一致。

8. 为什么 NoSQL 常采用 BASE 而不是 ACID？ NoSQL 面向高并发、海量数据和分布式扩展。在网络分区不可避免的情况下，严格 ACID 和强一致性可能降低可用性和扩展性。因此许多 NoSQL 系统采用 BASE 和最终一致性，在容忍短暂不一致的前提下提高可用性和性能。

本章易混点

NoSQL 不是完全不要 SQL。 更准确地说是 Not Only SQL，重点是非传统关系模型和分布式扩展。

Schema-less 不等于没有结构。 它只是不用像关系数据库那样提前固定所有列和表结构，应用层仍然要理解数据结构。

ACID 的一致性和 CAP 的一致性不同。 ACID 是事务合法状态；CAP 是分布式写后读一致。

CAP 不是“三选二系统随便选”。 在真实分布式系统中，P 通常必须考虑，所以主要是在 C 和 A 之间取舍。

最终一致性不是不一致。 它允许短暂不一致，但最终会收敛。

BASE 不是 ACID 的反面。 BASE 是在高可用分布式系统对强一致性的放松和工程妥协。

本章看到题目怎么写

看到**“NoSQL 是什么”**：写非关系型、Schema-less、集群化、多数据模型、高并发、海量数据、水平扩展。

看到**“四种 Data Model”**：写 Key-value、Document、Column-family、Graph，并各给一个适用场景。

看到**“ACID”**：写原子性、一致性、隔离性、持久性。

看到**“CAP”**：写 C/A/P 定义，三者不能同时完全满足，P 不可避免，常在 C 和 A 之间取舍。

看到**“ACID C vs CAP C”**：写事务一致性 vs 分布式读写一致性。

看到**“强一致 vs 最终一致”**：写立即看到最新值 vs 最后收敛；再举金融和点赞数例子。

看到**“BASE”**：写基本可用、软状态、最终一致性。

看到**“为什么 NoSQL 牺牲一致性”**：写网络分区不可避免，强一致会降低可用性，Web 场景常能容忍短暂不一致，所以采用最终一致性。